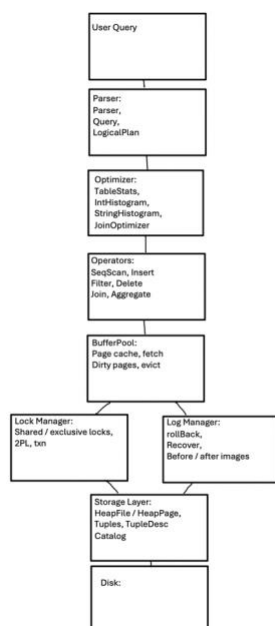


Section 1 / Part 1: Overview description

SimpleDB is organized as a layered relational database system in which higher-level query processing components ultimately rely on lower-level storage and buffer management components. At the bottom of the system are the classes that represent physical storage on disk, especially HeapFile and HeapPage, which store tuples in fixed-size pages and give the database a concrete access method for reading and writing relations. Above that sits the BufferPool, which is the main gateway for page access during execution. Operators do not read heap files directly during normal query processing, but instead request pages through the buffer pool, which caches pages in memory and later takes responsibility for eviction, dirty page handling, and transaction related page access. The catalog ties these pieces together by mapping table names and ids to the underlying storage files and schemas. The early project architecture described in Lab 1 already frames SimpleDB around tuples and schemas, access methods, operators, the buffer pool, and the catalog, and that structure remained the backbone of the final system. On top of the storage layer, SimpleDB uses an iterator-based operator model to execute query plans. Operators such as SeqScan, Filter, Join, Aggregate, Insert, and Delete implement relational algebra operations and are connected together into a tree, where leaf operators fetch tuples from storage and higher operators transform, combine, or summarize them. Lab 2 extends the Lab 1 storage engine into a functional relational engine by adding selection, join, aggregation, and table modification operators, while also expanding buffer-pool responsibilities to support page eviction. The result is that queries are executed by chaining operators together, with tuples flowing upward through the plan from storage level scans to the root operator that produces the final output. The later labs add transaction, recovery, and optimization services around that core execution engine. Lab 3 adds a locking-based transaction system using strict two-phase locking, so page accesses through the buffer pool become coordinated by lock management and transaction state. Lab 4 adds logging, rollback, and crash recovery, which turns the system from a basic transactional engine into one that can also recover from aborts and crashes using a log manager and before/after page images. Finally, Lab 5 adds a query optimizer on top of the existing parser and logical plan infrastructure by computing table statistics, histograms, scan costs, join costs, and join cardinality estimates. In the final system, a query therefore moves through several layers, the parser builds a logical plan, the optimizer and statistics modules help choose a physical plan, operators execute that plan, the buffer pool mediates page access, and the storage layer persists the actual tuples on disk.

Part 2: Diagram



Part 3: A more detailed description

The buffer manager in SimpleDB is centered around the BufferPool, which acts as the main interface between query execution and the pages stored on disk. Instead of operators reading heap files directly during normal execution, they request pages through the buffer pool. The buffer pool caches pages in memory, returns pages to callers, tracks dirty pages after updates, and handles page eviction when the cache is full. In the later labs, it also becomes the place where transaction related page access is coordinated, since locks are acquired before pages are returned and commit or abort behavior depends on the buffer pool's handling of modified pages. This makes the buffer pool one of the most important components in the system, because it connects query execution, storage, concurrency control, and recovery.

The operators are the execution engine of SimpleDB. They implement the iterator model of relational query processing, where each operator produces tuples for its parent operator. Core operators in the system include SeqScan, Filter, Join, and Aggregate, along with Insert and Delete for table modification. A query plan is therefore built as a tree of operators, with scans at the bottom reading tuples from storage and higher-level operators applying predicates, combining tuples, computing aggregates, or modifying tables. This design makes the system modular, because each operator is responsible for one relational task, while the overall query result is produced by composing operators together.

The lock manager is added in the transactions lab to provide isolation between concurrent transactions. Its job is to keep track of which transactions hold shared or exclusive locks on pages and to decide whether a new lock request can be granted immediately or must wait. Shared locks allow multiple readers, while exclusive locks are required for writes. The lock manager works with strict two-phase locking, which means that transactions acquire locks as needed during execution but do not release them until commit or abort. This prevents unsafe interleavings and ensures that concurrent transactions do not corrupt each other's results. In practice, the lock manager works closely with the buffer pool, since page requests flow through the buffer pool and lock checks happen before access is granted.

The log manager is introduced in the recovery lab and is responsible for supporting rollback and crash recovery. It records log entries containing before-images and after-images of updated pages so that the database can undo the effects of aborted transactions and redo the effects of committed ones after a crash. When a transaction aborts, the log manager uses the stored before-images to restore the correct earlier page contents. During recovery after a crash, it scans the log to determine which transactions were winners and which were losers, then redoes or undoes updates as needed. Together with the buffer pool and the locking layer, the log manager is what turns SimpleDB from a basic query engine into a transactional database system that can preserve correctness even in the presence of failures.

Part 5: Lab 5 answers

In Step 3, `simplifiedb.Parser.handleQueryStatement()`, the parser creates a `Query` object for the current transaction and then begins the planning process for the SQL query. It first calls `parseQueryLogicalPlan(tld, s)` to convert the parsed query into a `LogicalPlan`. After the logical plan has been created, `handleQueryStatement()` calls `lp.physicalPlan(tld, TableStats.getStatsMap(), explain)` to generate the physical execution plan. It then stores both the logical plan and the physical plan inside the `Query` object and returns that query. This is the key method in `Parser.java` where the query moves from a parsed SQL statement into SimpleDB's internal planning structures.

In Step 4, `simplifiedb.Parser.parseQueryLogicalPlan()`, the parser builds the `LogicalPlan` by translating the clauses of the SQL query into SimpleDB's logical representation. This method creates a new `LogicalPlan`,

records the query text, and then processes the query clause by clause. It adds scans for the tables in the FROM clause, processes WHERE expressions, and adds projected fields from the SELECT clause. If the query includes grouping, aggregation, or ordering, those are also added to the logical plan here. By the time this method finishes, the query has been fully translated into a logical plan that describes what operations need to be performed.

In Step 5, control returns to `handleQueryStatement()`, where the parser turns the logical plan into the physical plan by calling `lp.physicalPlan()`. This is the exact point where the executable operator tree is built. The result is an `OpIterator`, which is the physical plan that SimpleDB will actually run. `handleQueryStatement()` saves that physical plan in the Query object along with the logical plan, and then returns the completed Query back to the caller.

In Step 6, back in `simplifiedb.Parser.processNextStatement()`, the returned Query object is executed by calling `query.execute()`. This is the point where the finished physical plan actually runs and produces query results. If the statement was being run inside an automatically created transaction, `processNextStatement()` then commits that transaction after execution finishes. So the final part of the query's path through `Parser.java` is that `processNextStatement()` receives the planned query, executes it, and then commits if needed.

Section 2 / Discussion:

Overall, I think my final SimpleDB engine works well for what it is supposed to be, a small but complete educational database system. By the end of the project, it can store tuples on disk, execute relational operators, manage transactions with locking, recover from crashes using logging, and do basic cost estimation for query planning. What stands out to me most is that the system feels complete in the sense that all of the major pieces connect to each other. Queries are not just parsed and run in isolation but they flow through operators, depend on the buffer pool for page access, rely on locking for isolation, and use the log for recovery. At the same time, the engine is still clearly a simplified database system. It works, but it is not especially aggressive about performance, and many of the design choices favor clarity and correctness over efficiency.

If I had more time, the biggest thing I would improve is performance. I would want to build a more efficient join implementation, because joins are one of the most expensive parts of query execution and a basic nested-loops approach does not scale very well. I would also spend more time improving the buffer pool and eviction behavior, since memory management has a huge effect on the overall responsiveness of the system. Another area I would want to improve is deadlock handling and concurrency behavior, because even when correctness is there, performance can still suffer if transactions spend too much time waiting. On the recovery side, I would want to make the logging and rollback logic more robust and easier to reason about. Finally, I would push the optimizer further so it could make better decisions about join ordering and plan quality instead of stopping at the more basic cost and selectivity estimates. Overall, I feel good about the engine as a working database prototype, but if I had more time, I would focus on making it smarter, faster, and more scalable.